

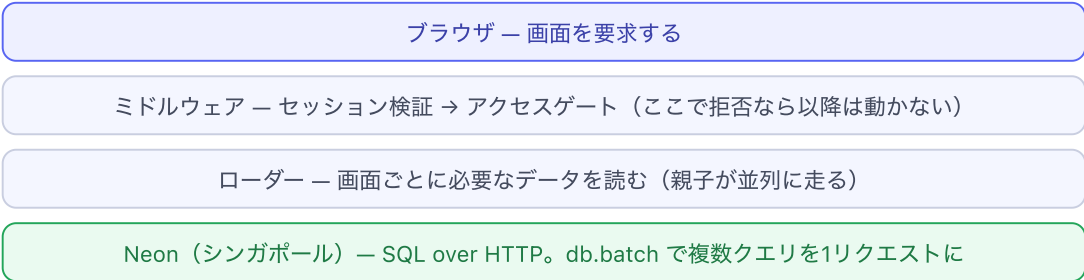
VECTA — 進捗レポート 2026-07-26

- 概要
- 仕組み
- 検証
- リスクと残タスク
- 用語集

用語集

Abstract

- What** このレポートに出てきた用語だけを、出てきた意味で説明する。
- Why** 用語を薄めて言い換えると正確さが落ちる。用語は残して、意味を足す。
- How** 「それが何か」と「なぜ必要か」を対にして書く。汎用の辞書は作らない。



context は、この縦の流れの中でデータを受け渡す入れ物

1リクエストが通る層。用語はどれもこの図のどこかに対応する。

Detail

1. リクエストの流れに関する用語

用語	それが何か / なぜ必要か
ラウンドトリップ (round trip)	プログラムがデータベースに問い合わせて答えが返るまでの、ネットワーク上の1往復。 なぜ重要か: 東京とシンガポールの間は光の速さでも往復に時間がかかり、クエリがどれだけ軽くても約70ms は消えない。だから「軽くする」より「回数を減らす」ほうが効く。
逐次 / 並列	前の答えを待ってから次を出すのが逐次、同時に出すのが並列。 なぜ重要か: 3回の問い合わせが並列なら待ち時間は1回ぶんだが、逐次だと3回ぶん積み上がる。今回の3本は完全に逐次だったので、1本消すと1本ぶんまるごと消える。

ミドルウェア (middleware)	ローダーより 前に 走る処理。 なぜ必要か : 「見てはいけない人のリクエストでは、子のローダーを1つも動かさない」を保証できるのはここだけ。親ローダーでは、子ローダーが並列で走ってしまうため間に合わない。
ローダー (loader)	画面を描く前にサーバー側で必要なデータを読む関数。 特徴 : 親レイアウトと子画面のローダーは 並列 に走る。今回、レイアウト側のローダーを軽くしたのはこの性質が理由。
context	1リクエストの間だけ生きる受け渡し用の入れ物。 なぜ必要か : ミドルウェアが確定した情報（誰か、どのプロジェクトのメンバーか）を、複数のローダーが取り出せるようにするため。リクエストごとに新品なので、他のリクエストに漏れない。

2. データベースに関する用語

用語	それが何か / なぜ必要か
SQL over HTTP	データベースへの問い合わせを、専用の接続ではなく通常の HTTP リクエストで送る方式。 なぜ必要か : Cloudflare Worker は1リクエストごとに起動と終了を繰り返すため、接続を確立するコスト（ハンドシェイク）を回収できない。HTTP なら確立するものが無い。
db.batch	複数のクエリを 1つの HTTP リクエスト にまとめて送る仕組み。サーバー側で1つのトランザクションとして実行される。 なぜ重要か : ワークスペース読み取りの8クエリが、ラウンドトリップ1回で済んでいるのはこれのおかげ。「クエリ数」と「往復回数」は別物である、という今回の話の前提。
複合キー (composite key)	1つの列ではなく複数の列の組み合わせで行を特定すること。ここでは (tenantId, projectId)。 なぜ必要か : プロジェクト ID だけで行を引けてしまうと、他社のプロジェクト ID を推測されたときに読めてしまう。テナントとの組で引くことで、構造的にそれを防ぐ。
ON DELETE CASCADE	親の行が消えたら、それに紐づく子の行も自動で消える設定。 ここでの意味 : プロジェクトを消すとメンバーシップも消えるので、「メンバーだがプロジェクトが無い」という状態は1リクエストの中でしか存在しない。

3. 今回の設計に固有の用語

用語	それが何か / なぜ必要か
アクセスゲート (access gate)	/projects/:id に入る全リクエストが必ず通る関門。ID の形式、そのユーザーが誰か、そのプロジェクトのメンバーかを順に確認し、駄目なら 404 で打ち切る。
fail closed	判断がつかないときは 拒否側に倒す 設計。 なぜ必要か : 逆 (fail open) だと、障害やデータ不整合のときに見えてはいけないものが見える。ここでは「メンバーではあるがプロジェクト行が読めない」を 404 にしている。

存在オラクル (existence oracle)	本来知られてはいけない「それが存在するかどうか」を、応答の違いから読み取れてしまう状態。 なぜ防ぐか : 「権限がありません(403)」と「ありません(404)」を区別できると、権限が無いのに 存在することだけ が分かってしまう。だから両方を、本文まで同一の 404 にしている。
記憶つき (memoise)	1度計算した結果を覚えておき、2度目以降は覚えた結果を返すこと。 なぜ必要か : 親レイアウトと子画面のローダーが並列に同じデータを求めても、問い合わせは1回で済む。
遅延 (lazy)	呼ばれるまで実行しないこと。 なぜ必要か : 拒否されたリクエストや、そのデータを使わないルート（素の <code>/projects/:id</code> ）が、問い合わせを1本も出さずに済む。
楽観的更新 (optimistic update)	サーバーの返事を待たずに画面を先に更新し、失敗したら巻き戻す方式。 ここでの関係 : 保存失敗時の応答は、この巻き戻しを駆動するための型付きデータになっている。だから 404 を throw に変えると巻き戻しが壊れる。
prefetch="intent"	リンクにマウスを載せた時点で、その先のデータを先読みする React Router の設定。 ここでの意味 : クリックしなくても読み取りが走るの、重いローダーの影響が想定より広く出る。

参考文献

1. React Router — Middleware / Loaders — reactrouter.com/how-to/middleware
2. Neon — Serverless driver — neon.com/docs/serverless/serverless-driver
3. OWASP — Improper access control / user enumeration — owasp.org/www-project-web-security-testing-guide
e